Lesson 7 — Timing and Counting: Hardware Timers and Counters

ATmega328P / Arduino Uno | Source: lecture 07 slides (Systems 10–13), Exercise 5 | Builds on Lessons 3–4 (ports, polling, interrupts)

WHAT YOU WILL BE ABLE TO DO AFTER THIS LESSON

- Explain **why** we use hardware timers and counters instead of software delay loops and software counters.
- Explain how a timer counts, what **overflow** and the **TOV flag** are, and how elapsed time is calculated.
- Use `TCCRnB`, `TCNTn`, `TIFRn` and `TIMSKn` to **count events** on the T0/T1 pin and to make **precise delays**.
- Calculate the **initial (preload) value** for any count or any delay, by polling or with an interrupt.

GIVEN TO YOU IN THE EXAM

The register sheet shows the bit names and positions of the timer registers (`TCCR0B`, `TCCR1B`, `TIFR0/1`, `TIMSK0/1`) and the Arduino pin sheet shows that **D4 = PD4 = T0** and **D5 = PD5 = T1**. Check your printed copies. **What the CS bit codes mean, the formulas and the TOV rules below are "learn it".**

1. Why hardware timers and counters?

A microcontroller is a **single-chip computer**: one CPU plus **peripherals** on the same chip. You already used two peripherals: the **ports** (Lesson 3) and the **ADC** (Lesson 5). The **timer/counter** is a third one. It is a separate piece of hardware that counts pulses **by itself**, while the CPU does something else.

Two jobs come up again and again:

- **Timing**: measure the time between two events, or wait a precise time (blink every 200 ms).
- **Counting**: count events, for example items passing on a conveyor belt, or people crossing an infrared beam at a door.

Both can be done in **software**, which is what you did so far:

```
// software timing: 1 minute
int i = 0;
while (1) {
    _delay_ms(1000);   // CPU spins 1 s
    i++;
    if (i == 60) { /* 1 minute */ }
}
```

```
// software counting: people
int count = 0;
while (1) {
    if (beam sensor is LOW)
        count++;       // CPU watches
}
```

The problem: a delay loop is a **busy wait**. The CPU runs an empty loop just to burn time, so it **cannot do anything else**. The ATmega328P has **one core**, so that time is completely wasted. (On a PC you don't notice this, because a multi-core processor runs other work on the other cores.)

	Software (delay loop / variable)	Hardware timer / counter
CPU while timing/counting	tied up (busy wait or constant checking)	**free** to do another task
Precision	approximate (depends on the code around it)	**exact**: counts crystal clock pulses
When to use	quick tests	anything time-critical; the efficient way

2. Timer or counter? Same hardware, different clock source

Inside, a timer is just a **register that adds 1 on every pulse it receives**. Where the pulses come from decides its name:

- Pulses from the **internal oscillator** (the 16 MHz crystal, optionally slowed by the **prescaler**): the pulses arrive at a known rate, so the count tells you **time**. It works as a **timer**.
- Pulses from an **external source on the Tn pin** (a button, an IR beam sensor, a door switch): each pulse is one event, so the count tells you **how many events**. It works as a **counter**.

The choice is made by the three **clock select (CS) bits** in `TCCRnB`. The *n* is the timer number: 0, 1 or 2.

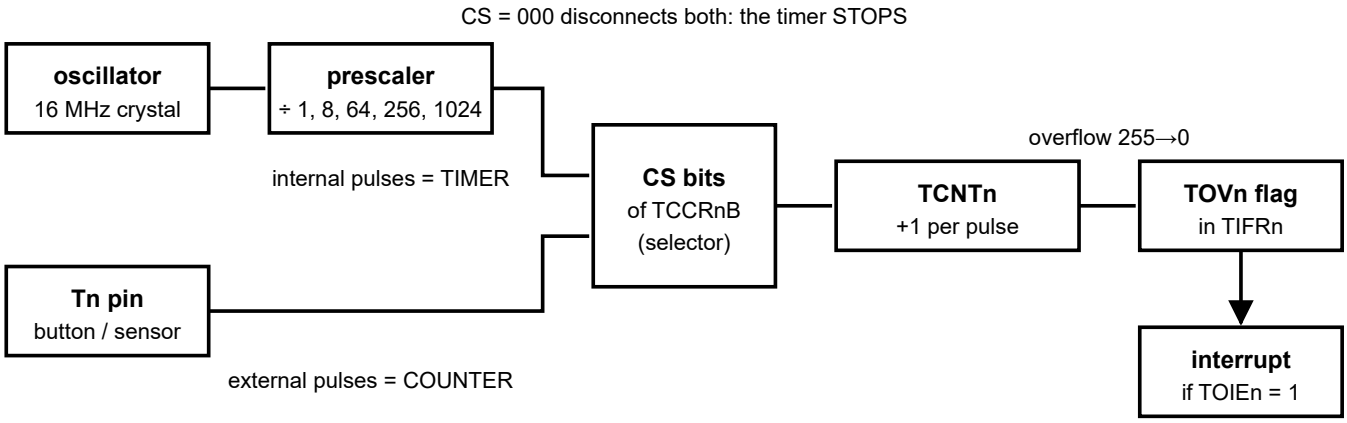


Figure 1. One timer/counter. The CS bits choose where the pulses come from; TCNTn counts them; on overflow the TOV flag is raised.

3. How the count register works: count, overflow, TOV

`TCNTn` (Timer/CouNTer register) goes up by 1 on every pulse. It has a fixed size:

- **Timer0 and Timer2 are 8-bit**: TCNT0 counts 0, 1, 2 … **255**, then the next pulse makes it **overflow back to 0**.
- **Timer1 is 16-bit**: TCNT1 counts 0 … **65 535** ($2^{16} - 1$), then overflows to 0.

At the moment of the overflow the hardware sets the **TOV** (Timer OVerflow) **flag**, bit 0 of `TIFRn` (Timer Interrupt Flag Register). The flag says: *"I counted up to my maximum and started again from 0."*

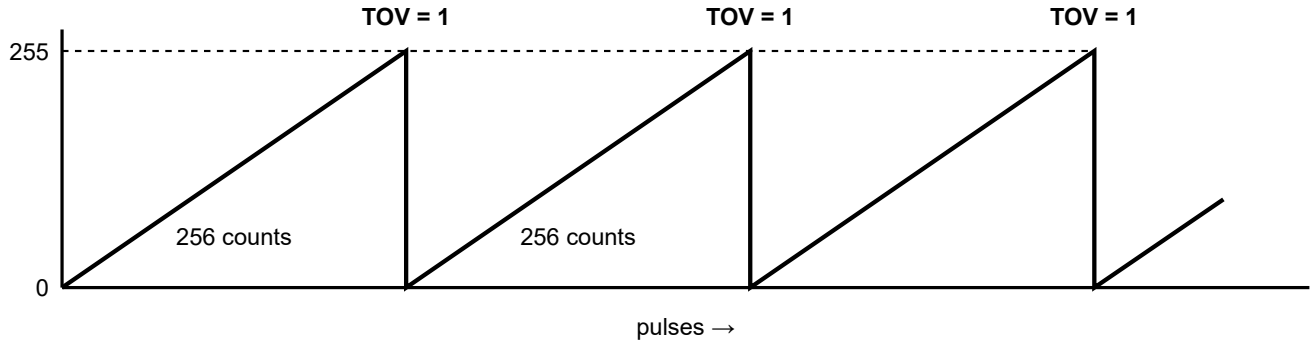


Figure 2. TCNT0 climbs 0→255, overflows to 0 and starts again. Each overflow raises the TOV flag. (TCNT1 does the same up to 65 535.)

THE FLAG RULE: WRITE 1 TO CLEAR [VERY IMPORTANT]

A **flag bit is set by the hardware**, never by you. You can only **clear** it, and you clear it by **writing a 1 to it** (writing 0 does nothing). If you poll the flag you **must clear it after each overflow**, otherwise it stays 1 and the hardware cannot raise it again for the next overflow.

```
TIFR0 = TIFR0 | 0b00000001;   // write 1 to TOV0 = CLEAR it (slide style)
TIFR0 = (1 << TOV0);          // same effect, touches only TOV0
```

When you use the **overflow interrupt** instead, the hardware clears TOV **automatically** when the ISR runs, so the ISR does not clear it.

4. Measuring elapsed time

The oscillator gives *f* pulses per second. If TCNT started at **A** and you stop the timer when it shows **B**, then B − A pulses passed, each lasting 1/*f* second:

ELAPSED TIME

without prescaler: **t = (B − A) / f** with prescaler N: **t = N × (B − A) / f**

If it overflowed k times in between, add k × 256 (8-bit) or k × 65 536 (16-bit) pulses. You keep k in a variable.

Example: f = 1 Hz, A = 0, B = 60 → t = 60 s. With f = 10 Hz and prescaler 10, the timer gets 1 pulse per second, so again A = 0, B = 60 gives 10 × 60 / 10 = 60 s.

Why the prescaler? At 16 MHz an 8-bit timer fills up in 256 / 16 000 000 = 16 µs. That is far too fast. The prescaler divides the clock by N so the register climbs slowly enough to measure useful times. External pulses (buttons, sensors) are already slow, so there is **no prescaler on the Tn pin**.

Stopping the timer: set the CS bits to **000**. That disconnects every clock source, TCNT freezes, and you can read it.

5. The timer registers bit by bit

`TCCR0B` / `TCCR1B` **(Timer/Counter Control Register B): the CS bits**

	7	6	5	4	3	2	1	0	
TCCRnB	–	–	–	–	–	**CS2**	**CS1**	**CS0**	only bits 2–0 matter in this lesson

CS2 CS1 CS0	Clock source	Mode	One tick at 16 MHz
`000`	none: timer **stopped**	–	–
`001`	F_CPU / 1	timer	0.0625 µs
`010`	F_CPU / 8	timer	0.5 µs
`011`	F_CPU / 64	timer	4 µs
`100`	F_CPU / 256	timer	16 µs
`101`	F_CPU / 1024	timer	64 µs
`110`	Tn pin, **falling edge** (HIGH→LOW)	counter	–
`111`	Tn pin, **rising edge** (LOW→HIGH)	counter	–

Falling or rising? A button on a pin with the pull-up ON reads HIGH and goes LOW when pressed. **Falling edge (110) counts presses**; **rising edge (111) counts releases**.

`TIFR0` / `TIFR1` **(Timer Interrupt Flag Register) and** `TIMSK0` / `TIMSK1` **(Timer Interrupt MaSK)**

TIFRn	–	–	–	–	–	·	·	**TOV**	bit 0: set by hardware on overflow; write 1 to clear
TIMSKn	–	–	–	–	–	·	·	**TOIE**	bit 0: 1 = overflow interrupt enabled (default 0)

Timer	Size	Count register	External pin	Overflow ISR vector
Timer0	8-bit	`TCNT0` (0–255)	T0 = PD4 = **D4**	`TIMER0_OVF_vect`
Timer1	16-bit	`TCNT1` (0–65 535)	T1 = PD5 = **D5**	`TIMER1_OVF_vect`
Timer2	8-bit	`TCNT2`	TOSC pins = PB6/PB7: **used by the crystal** on the Uno	`TIMER2_OVF_vect`

So on the Uno you can count external pulses only with **Timer0 (D4)** or **Timer1 (D5)**. Timer2 can still be a timer from the internal clock.

6. The preload formula (for counts and for delays)

The flag is raised only on the overflow. To get the flag after exactly **n** pulses, start the register **n pulses before the overflow**:

INITIAL VALUE

initial = max − n + 1 (max = 255 for 8-bit, 65 535 for 16-bit; the **+1 is the step from max to 0**, the overflow itself)

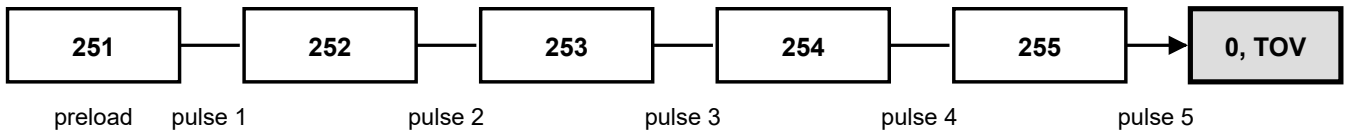


Figure 3. Count 5 with Timer0: 255 − 5 + 1 = 251. The 5th pulse causes the overflow and raises TOV.

For a **delay**, first find how many timer pulses n the delay needs:

DELAY CALCULATION

one tick = prescale / F_CPU ⇒ delay = (prescale / F_CPU) × n ⇒ **n = (F_CPU / prescale) × delay** (delay in **seconds**)

then **initial = max − n + 1**. n must be ≤ 256 (8-bit) or ≤ 65 536 (16-bit).

Worked example	n	Fits?	Initial value
200 ms, prescale 64 (slides)	16 000 000 / 64 × 0.2 = **50 000**	not 8-bit; Timer1 yes	65 535 − 50 000 + 1 = **15 536**
1 s, prescale 1024 (Exercise 5 Q2)	16 000 000 / 1024 × 1 = **15 625**	Timer1	65 535 − 15 625 + 1 = **49 911**
400 ms, prescale 64	100 000	**no** (> 65 536)	use /256: n = 25 000 → **40 536**

Longest single-overflow delay at 16 MHz	/1	/8	/64	/256	/1024
8-bit (Timer0)	16 µs	128 µs	1.024 ms	4.096 ms	16.384 ms
16-bit (Timer1)	4.096 ms	32.768 ms	262.1 ms	1.049 s	4.194 s

Longer than that? Count overflows in a variable (e.g. ten 1-s overflows = 10 s), or use a larger prescaler. There is no "best" prescaler: pick one that makes n fit and gives whole numbers.

7. Counting events: Systems 10–12 (button on T0 counts 5 presses, LED toggles)

Hardware (same for all three): pushbutton from **D4 (PD4 = T0)** to GND, internal pull-up ON, a **100 nF capacitor across the button** to debounce it (each bounce would otherwise be counted as a press), and the on-board LED on **D13 (PB5)**.

System 10 — poll the count

```
#include <avr/io.h>

int main(void)
{
    DDRB = DDRB | 0b00100000;        // PB5 output (LED)
    DDRD = DDRD & 0b11101111;        // PD4 (T0) input (already 0 by default)
    PORTD = PORTD | 0b00010000;      // pull-up on PD4
    TCCR0B = TCCR0B | 0b00000110;    // CS = 110: clock = T0 pin, falling edge -> COUNTER
    TCNT0 = 0;                       // start count (already 0 by default)
    while (1)
    {
        if (TCNT0 >= 5)              // 5 presses counted by HARDWARE
        {
            PORTB = PORTB ^ 0b00100000;   // toggle LED
            TCNT0 = 0;                    // restart count
        }
    }
}
```

Why `>= 5` and not `== 5`? If two pulses arrive between two checks (a spike the capacitor missed, or a very fast double press), TCNT0 jumps from 4 to 6. With `==` the LED would then need 250 more presses (round to 255, overflow, back to 5). `>=` still catches it.

System 11 — exactly 5 presses: preload 251 and poll TOV

```
    TCNT0 = 251;                        // 255 - 5 + 1
    while (1)
    {
        if (TIFR0 & 0b00000001)         // TOV0 set? (5th press caused overflow)
        {
            PORTB = PORTB ^ 0b00100000;  // toggle LED
            TCNT0 = 251;                 // reload (NOT 0)
            TIFR0 = TIFR0 | 0b00000001;  // clear TOV0 by writing 1
        }
    }
```

How the `if` works: AND with `0b00000001` forces bits 7–1 to 0 and keeps bit 0 as it is. The result is 1 (true) only when TOV0 = 1.

System 12 — exactly 5 presses with the overflow interrupt

```
#include <avr/io.h>
#include <avr/interrupt.h>

int main(void)
{
    DDRB = DDRB | 0b00100000;        // PB5 output
    DDRD = DDRD & 0b11101111;        // PD4 input
    PORTD = PORTD | 0b00010000;      // pull-up PD4
    TCCR0B = TCCR0B | 0b00000110;    // clock = T0, falling edge
    TCNT0 = 251;                     // overflow after 5 presses
    TIMSK0 = TIMSK0 | 0b00000001;    // TOIE0 = 1: enable overflow interrupt
    sei();                           // enable global interrupts
    while (1)
    {
        // CPU is free for another task
    }
}

ISR(TIMER0_OVF_vect)                 // runs automatically on overflow
{
    PORTB = PORTB ^ 0b00100000;      // toggle LED
    TCNT0 = 251;                     // reload (TOV0 is cleared by hardware)
}
```

8. Precise delays: System 13 (blink the LED every 200 ms with Timer1)

No external hardware: on-board LED on PB5. From section 6: prescale 64, n = 50 000, initial = **15 536**.

Polling TOV1

```
#include <avr/io.h>

int main(void)
{
    DDRB = DDRB | (1<<DDB5);
    TCCR1B = TCCR1B | (1<<CS11)
                    | (1<<CS10); // /64
    TCNT1 = 15536;
    while (1)
    {
        if (TIFR1 & (1<<TOV1))
        {
            PORTB = PORTB ^ (1<<PORTB5);
            TCNT1 = 15536;
            TIFR1 = TIFR1 | (1<<TOV1);
        }
    }
}
```

Overflow interrupt

```
#include <avr/io.h>
#include <avr/interrupt.h>

int main(void)
{
    DDRB = DDRB | (1<<DDB5);
    TCCR1B = TCCR1B | (1<<CS11)
                    | (1<<CS10); // /64
    TCNT1 = 15536;
    TIMSK1 = TIMSK1 | (1<<TOIE1);
    sei();
    while (1) { }   // free
}

ISR(TIMER1_OVF_vect)
{
    PORTB = PORTB ^ (1<<PORTB5);
    TCNT1 = 15536;
}
```

These are the slides' "labeled constants" versions: `(1<<CS11)` means "a 1 shifted to the CS11 position" = `0b00000010`. Same result as the binary strings, easier to read. The binary-string versions use `TCCR1B | 0b00000011` and `TIFR1 & 0b00000001`.

9. Build example: Exercise 5 — the reaction timer

Task: press **READY** → all LEDs off, wait a **random 1–10 s** → **blue** LED on → user presses **REACT** as fast as possible → blue off, show the reaction time: **green** < 1 s, **orange** 1 s to < 2 s, **red** ≥ 2 s.

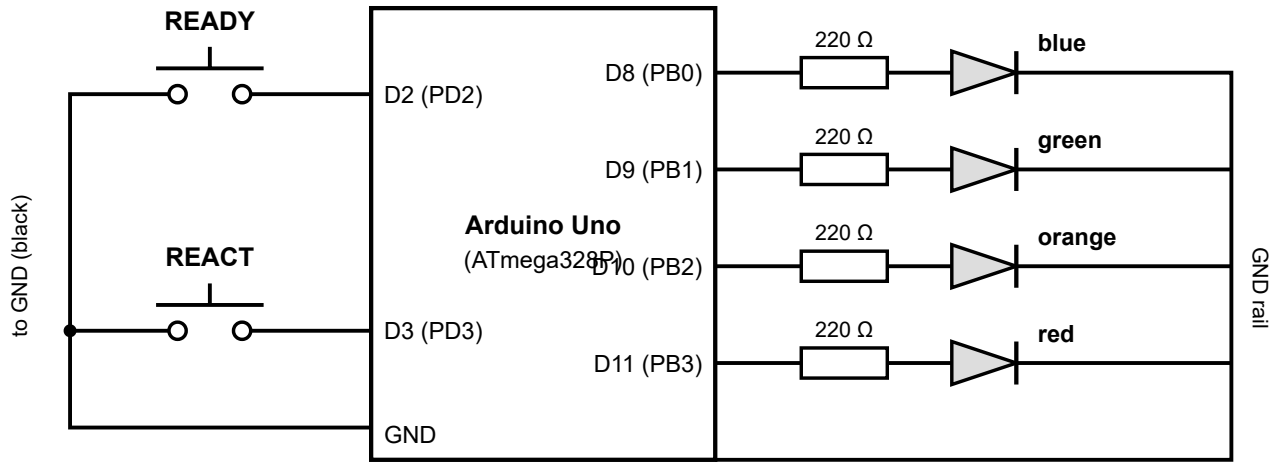


Figure 4. Reaction timer. Buttons to GND with internal pull-ups (pressed = LOW). Each LED: pin → 220 Ω → anode (long leg), cathode (flat side) → GND.

Part	Pin	Why this choice
READY button	D2 / PD2	any input; pull-up so no resistor needed
REACT button	D3 / PD3	same
LEDs blue, green, orange, red	D8–D11 / PB0–PB3	all on port B, so one register controls them
Timer0 (8-bit)	–	runs freely at 16 MHz; its value when READY is pressed is the random **seed** (human timing is random)
Timer1 (16-bit)	–	/1024 ⇒ 1 tick = 64 µs. Preload 49 911 = 1-s overflows for the wait; from 0 it measures up to 4.19 s for the reaction

```
#include <avr/io.h>
#include <stdlib.h>                    // srand(), rand()

int main(void)
{
    DDRB = DDRB | 0b00001111;           // PB0-PB3 outputs: blue, green, orange, red
    DDRD = DDRD & 0b11110011;           // PD2 READY, PD3 REACT inputs
    PORTD = PORTD | 0b00001100;         // pull-ups on PD2, PD3
    TCCR0B = TCCR0B | 0b00000001;       // Timer0 free-running at F_CPU/1 (seed source)
    TCCR1B = TCCR1B | 0b00000101;       // Timer1 at F_CPU/1024: 1 tick = 64 us

    while (1)
    {
        while (PIND & 0b00000100) { }   // wait until READY pressed (PD2 LOW)
        srand(TCNT0);                     // fairly random seed
        while (!(PIND & 0b00000100)) { }  // wait until READY released
        PORTB = PORTB & 0b11110000;       // all LEDs off

        unsigned char waitSec = rand() % 10 + 1;   // 1..10 s
        unsigned char sec = 0;
        TCNT1 = 49911;                    // 65535 - 15625 + 1: overflow after 1 s
        TIFR1 = TIFR1 | 0b00000001;       // clear old TOV1
        while (sec < waitSec)             // count 1-s overflows
        {
            if (TIFR1 & 0b00000001)
            {
                sec++;
                TCNT1 = 49911;
                TIFR1 = TIFR1 | 0b00000001;
            }
        }

        PORTB = PORTB | 0b00000001;       // blue ON: react now
        TCNT1 = 0;                        // start measuring
        TIFR1 = TIFR1 | 0b00000001;
        while ((PIND & 0b00001000) && !(TIFR1 & 0b00000001)) { }  // wait for REACT or 4.19 s
        unsigned int ticks = TCNT1;       // 1 tick = 64 us
        PORTB = PORTB & 0b11111110;       // blue OFF

        if ((TIFR1 & 0b00000001) || ticks >= 31250)
            PORTB = PORTB | 0b00001000;   // red: 2 s or more (31250 x 64 us = 2 s)
        else if (ticks >= 15625)
            PORTB = PORTB | 0b00000100;   // orange: 1 s to < 2 s
        else
            PORTB = PORTB | 0b00000010;   // green: under 1 s
    }
}
```

Q4: SEED FROM A FLOATING ANALOG INPUT

Leave A0 unconnected. Its reading drifts randomly (noise). Read it once with the ADC (Lesson 5: `ADMUX` = AVcc ref, channel 0; `ADCSRA` enable + start; wait for ADSC = 0) and use `srand(ADCW);` instead of `srand(TCNT0);`. Then Timer0 is no longer needed.

Q5–Q6: ARDUINO VERSION AND COMPARISON

`randomSeed(analogRead(A0));` `random(1000, 10001)` ms wait with `delay()`; reaction with `millis()`; `digitalRead/digitalWrite`.

Compare: Arduino is shorter and easier to read, but larger and slower, its `delay()` busy-waits, and the platform gives **no direct timer access**. The C version uses the hardware timers directly: precise, small, fast, CPU free.

L. What the lecturer explained in the lectures (2020 recordings)

From the two timing-and-counting recordings (folders 20–21 in `Embedded Systems Lectures`). **[EXAM]** marks things he tied to the exam.

L1. WHY HE WANTS THE INTERNALS [EXAM]

"Many people know how to **use** timers, but if you ask them to explain the **internal working**, they don't know." As university students you are expected to explain how a timer works (Figure 1 and Figure 2), not just call it. He also said some written questions are set to check whether you **think beyond what was taught**, which is why the module has a reputation for being hard.

L2. Where timers fit

- So far: inputs (digital via ports, analog via ADC) and driving digital actuators (directly, or via BJT / MOSFET / relay). Analog actuators come next with **PWM**.
- A microcontroller is a "single-chip computer": processor + peripherals. Ports and ADC are peripherals; the timer is another.
- Small chips (ATtiny) have 1–2 timers named T0, T1…; ATmega has 3–5; XMega even more. "Large" means capability, not size: many big microcontrollers are tiny SMD (surface-mount) packages that are **not breadboard friendly**.

L3. Software vs hardware timing, in his words

- A delay loop is a **busy wait**: "we tie up the CPU to run a loop but doing nothing", just to consume time. That CPU time could have been used for something else.
- On PCs you never feel this because of multi-core processors; the microcontroller has **one core**, so it matters.
- "This is the first time you are going to use a hardware timer." Two advantages: **CPU is free** during the delay, and hardware timing is **highly precise**. "If the system is a critical system which depends on time measurements, you better use hardware timers."
- The same applies to counting: a software counter (`if (sensor LOW) i++;`) ties up the CPU.

L4. Timer becomes counter

- Internal clock connected = timer. "The timer becomes a counter when an external pulse source is connected."
- His examples of external pulses: an **IR beam** across a lecture-hall entrance (beam present = HIGH, person crossing = LOW, so people produce a square wave); items on a **conveyor belt**; and the cheapest one, a **push button between a door and its frame** with an automatic door closer. Count 5 openings, then drive a **solenoid** through a transistor to lock the door: "we allowed only five people". He said this is the kind of thing that could be a practical exam question.
- Elapsed time worked example: f = 1 Hz, count 100 ⇒ 100 s. Fast oscillator + small register ⇒ fills too quickly ⇒ use the prescaler.
- Student asked: is there a prescaler for the external source? **No**: external sources are always slow compared with the CPU.

L5. Flag bits [EXAM]

- "Flag bit means: if something happens, you raise the flag to indicate", like a white flag to surrender.
- **"To clear a flag bit, set it."** Writing 1 makes it 0; writing 0 does nothing. The hardware sets flags, you only clear them. This is standard on many architectures, not only AVR (he named Intel and Motorola, now NXP).
- To know whether a bit is a flag bit, **check the datasheet**.
- When you use the overflow **ISR**, the flag is cleared by hardware automatically; only when polling must you clear it yourself.

L6. Details from Systems 10–12

- Pins: **T0 = PD4 (Arduino 4)**, **T1 = PD5 (Arduino 5)**. Timer2's TOSC pins (PB6/PB7) already have the 16 MHz crystal, "so we cannot use Timer2" for an external clock.
- The on-board LED uses a **1 kΩ** resistor because an SMD LED needs very little current. The **100 nF capacitor in parallel with the button** debounces the input. Place buttons **diagonally** on the breadboard.
- `DDRD &= ...` for PD4 and `TCNT0 = 0` are **not necessary** (both are 0 after reset), but "make your algorithm more clear". The cost is one extra clock cycle each.
- **Why `>=` not `==` [EXAM-style reasoning]:** a spike the capacitor misses, or a very fast double press, can take the count from 4 straight to 6. With `==` you'd have to press until 255, overflow and come back to 5. The chance is slim, but "as programmers you have to consider all possible situations", especially in mission-critical, safety-critical, medical, military or aerospace systems.

- If the client requires **exactly** 5 presses, use the preload 251 + TOV method (System 11). Formula: max − count + 1; "plus 1 to account for the overflow". Reset means reload **251, not 0**.
- Polling TOV still wastes the CPU, so the best approach is the overflow interrupt (System 12). "This is the most efficient way of using counters."
- Arduino platform: **no direct access to timers** and **no wrapper for timer interrupts** (`attachInterrupt` is only for external interrupts). Some Arduino libraries use timers internally, but you can't. This is another reason for C.
- A student asked whether Exercise 3 could use counters. Yes, but he expected Exercise 3 to be done without them.

L7. Delays with Timer1 (System 13)

- Derivation: f = 1/t, so one tick = 1 / (F_CPU / prescale) = prescale / F_CPU. Then delay = tick × n, so n = F_CPU / prescale × delay. **Give the delay in seconds** (200 ms = 200 × 10^{-3}).
- 200 ms at /64 ⇒ n = 50 000: "can you put it into an 8-bit timer? No." So use Timer1 (16-bit). Initial = 65 535 − 50 000 + 1 = **15 536**.
- 400 ms needs 100 000 counts, too many even for 16 bits. Three fixes he listed: **(1)** a bigger prescaler (256, 1024; but even 1024 has a limit), **(2)** a variable counting overflows, **(3)** in your own hardware, a slower crystal (8 or 4 MHz).
- "Is /64 the optimal prescale?" **No optimal value**: "I just use 64, because I like 64." The prescaler scales the CPU clock down to suit the job (the ADC needs it too).
- He explained `if (TIFR1 & 0b00000001)` bit by bit: x AND 0 = 0 for bits 7–1; x AND 1 = x for bit 0, so the condition is true only when TOV1 = 1.

L8. MARKS AND THE PRACTICAL EXAM [EXAM]

50% practical, 50% written (most other modules are about 70 / 25 / 25). In the practical, within **3 hours** you design the hardware, assemble it, design and implement the firmware, run it and show it. If you score only 25/50 in the practical, you need 45/50 (90%) in the written paper for an A, which is very hard. "I know all the theories about swimming, but I can't swim": practise with your own kit. When doing exercises, also try **other ways** of solving the same exercise.

10. Common mistakes

Mistake	What happens / fix
Forgetting to clear TOV when polling	Flag stays 1, the `if` is true every loop. Write 1 to it after handling.
Clearing TOV by writing 0	Nothing happens. Flags clear by writing **1**.
Clearing TOV inside the ISR	Not needed (hardware does it), but harmless.
Reloading 0 instead of the preload	Next period counts 256 / 65 536 pulses, not n.
Delay in ms in the formula	Use seconds: n = F_CPU / prescale × 0.2, not × 200.
n bigger than the register	n > 256 needs Timer1; n > 65 536 needs a bigger prescaler or an overflow counter.
Forgetting `sei()` or TOIE	ISR never runs. You need both, plus `#include <avr/interrupt.h>`.
Wrong vector name	Timer0 = `TIMER0_OVF_vect`, Timer1 = `TIMER1_OVF_vect`.
Button on the wrong pin for counting	External counting only works on **D4 (T0)** or **D5 (T1)**.
No pull-up / no debounce on T0	Floating pin counts noise; bounces count as many presses.

Cheat sheet

Registers	
TCCRnB CS2..0	000 stop, 001 /1, 010 /8, 011 /64, 100 /256, 101 /1024, 110 Tn fall, 111 Tn rise
TCNTn	count; T0, T2 8-bit (255), T1 16-bit (65 535)
TIFRn bit 0	TOV: set on overflow, write 1 to clear
TIMSKn bit 0	TOIE: overflow interrupt enable (+ sei)
Pins	T0 = D4 (PD4), T1 = D5 (PD5)

Formulas	
Tick	prescale / F_CPU
n	F_CPU / prescale × delay (s)
Initial	max − n + 1
Elapsed	N (B − A) / f
200 ms	/64, Timer1 → 15 536
1 s	/1024, Timer1 → 49 911
Count 5	Timer0 → 251